

Python - Dictionaries

Dictionaries in Python

In Python, a dictionary is a built-in data type that stores data in key-value pairs. It is an unordered, mutable, and indexed collection. Each key in a dictionary is unique and maps to a value. Dictionaries are often used to store data that is related, such as information associated with a specific entity or object, where you can quickly retrieve a value based on its key.

Python's dictionary is an example of a mapping type. A mapping object 'maps' the value of one object to another. To establish mapping between a key and a value, the colon (:) symbol is put between the two.

Each key-value pair is separated by a comma and enclosed within curly braces {}. The key and value within each pair are separated by a colon (:), forming the structure key:value.

Given below are some examples of Python dictionary objects –

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",  
            "Telangana":"Hyderabad", "Karnataka":"Bengaluru"}  
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty", 40:"Forty"}  
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
```

Key Features of Dictionaries

Following are the key features of dictionaries –

- **Unordered** – The elements in a dictionary do not have a specific order. Python dictionaries before version 3.7 did not maintain insertion order. Starting from

Python 3.7, dictionaries maintain insertion order as a language feature.

- **Mutable** – You can change, add, or remove items after the dictionary has been created.
- **Indexed** – Although dictionaries do not have numeric indexes, they use keys as indexes to access the associated values.
- **Unique Keys** – Each key in a dictionary must be unique. If you try to assign a value to an existing key, the old value will be replaced by the new value.
- **Heterogeneous** – Keys and values in a dictionary can be of any data type.

Example 1

Only a number, string or tuple can be used as key. All of them are immutable. You can use an object of any type as the value. Hence following definitions of dictionary are also valid –

```
d1 = {"Fruit":["Mango","Banana"], "Flower":["Rose", "Lotus"]}
d2 = {('India, USA'):'Countries', ('New Delhi', 'New York'):'Capitals'}
print (d1)
print (d2)
```

It will produce the following **output** –

```
{'Fruit': ['Mango', 'Banana'], 'Flower': ['Rose', 'Lotus']}
{'India, USA': 'Countries', ('New Delhi', 'New York'): 'Capitals'}
```

Example 2

Python doesn't accept mutable objects such as list as key, and raises TypeError.

```
d1 = {["Mango","Banana"]:"Fruit", "Flower":["Rose", "Lotus"]}
print (d1)
```

It will raise a TypeError –

```
Traceback (most recent call last):  
  File "C:\Users\Sairam\PycharmProjects\pythonProject\main.py", line 8, in <module>  
    d1 = [{"Mango","Banana"}:"Fruit", "Flower":["Rose", "Lotus"]}  
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
TypeError: unhashable type: 'list'
```

Example 3

You can assign a value to more than one keys in a dictionary, but a key cannot appear more than once in a dictionary.

It will produce the following **output** –

```
{'Banana': 'Fruit', 'Rose': 'Flower', 'Lotus': 'Flower', 'Mango': 'Fruit'}
{'Fruit': 'Mango', 'Flower': 'Lotus'}
```

Creating a Dictionary

You can create a dictionary in Python by placing a comma-separated sequence of key-value pairs within curly braces {}, with a colon : separating each key and its associated value. Alternatively, you can use the dict() function.

Example

The following example demonstrates how to create a dictionary called "student_info" using both curly braces and the dict() function –

```
# Creating a dictionary using curly braces
sports_player = {
```

```

    "Name": "Sachin Tendulkar",
    "Age": 48,
    "Sport": "Cricket"
}
print ("Dictionary using curly braces:", sports_player)
# Creating a dictionary using the dict() function
student_info = dict(name="Alice", age=21, major="Computer Science")
print("Dictionary using dict():", student_info)

```

The result produced is as shown below –

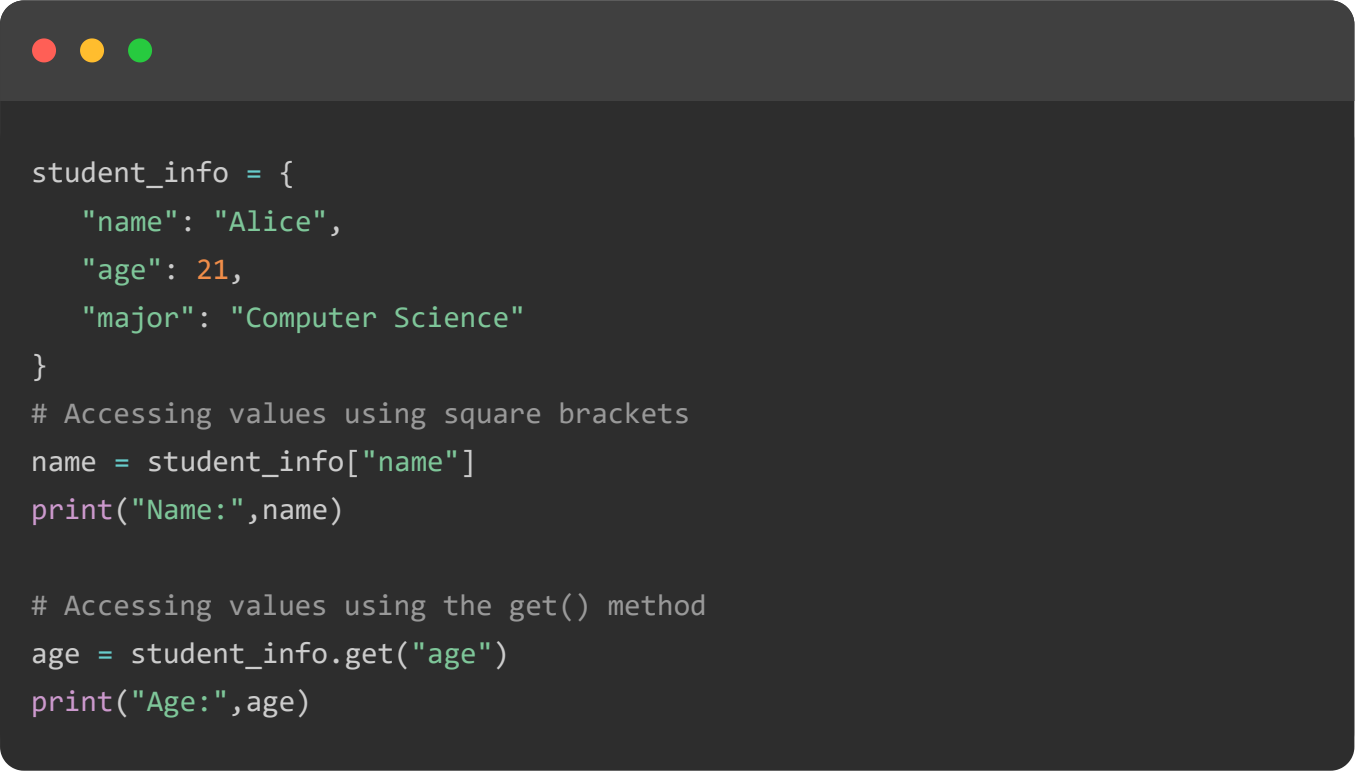
```

Dictionary using curly braces: {'Name': 'Sachin Tendulkar', 'Age': 48, 'Sport':
Dictionary using dict(): {'name': 'Alice', 'age': 21, 'major': 'Computer Science

```

Accessing Dictionary Items

You can access the value associated with a specific key using square brackets [] or the get() method –



```

student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Accessing values using square brackets
name = student_info["name"]
print("Name:", name)

# Accessing values using the get() method
age = student_info.get("age")
print("Age:", age)

```

The result obtained is as follows –

```

Name: Alice
Age: 21

```

Modifying Dictionary Items

You can modify the value associated with a specific key or add a new key-value pair –

```
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Modifying an existing key-value pair
student_info["age"] = 22

# Adding a new key-value pair
student_info["graduation_year"] = 2023
print("The modified dictionary is:", student_info)
```

Output of the above code is as follows –

```
The modified dictionary is: {'name': 'Alice', 'age': 22, 'major': 'Computer Science', 'graduation_year': 2023}
```

Removing Dictionary Items

You can remove items using the `del` statement, the `pop()` method, or the `popitem()` method –

```
student_info = {
    "name": "Alice",
    "age": 22,
    "major": "Computer Science",
    "graduation_year": 2023
}
# Removing an item using the del statement
del student_info["major"]

# Removing an item using the pop() method
```

```
graduation_year = student_info.pop("graduation_year")

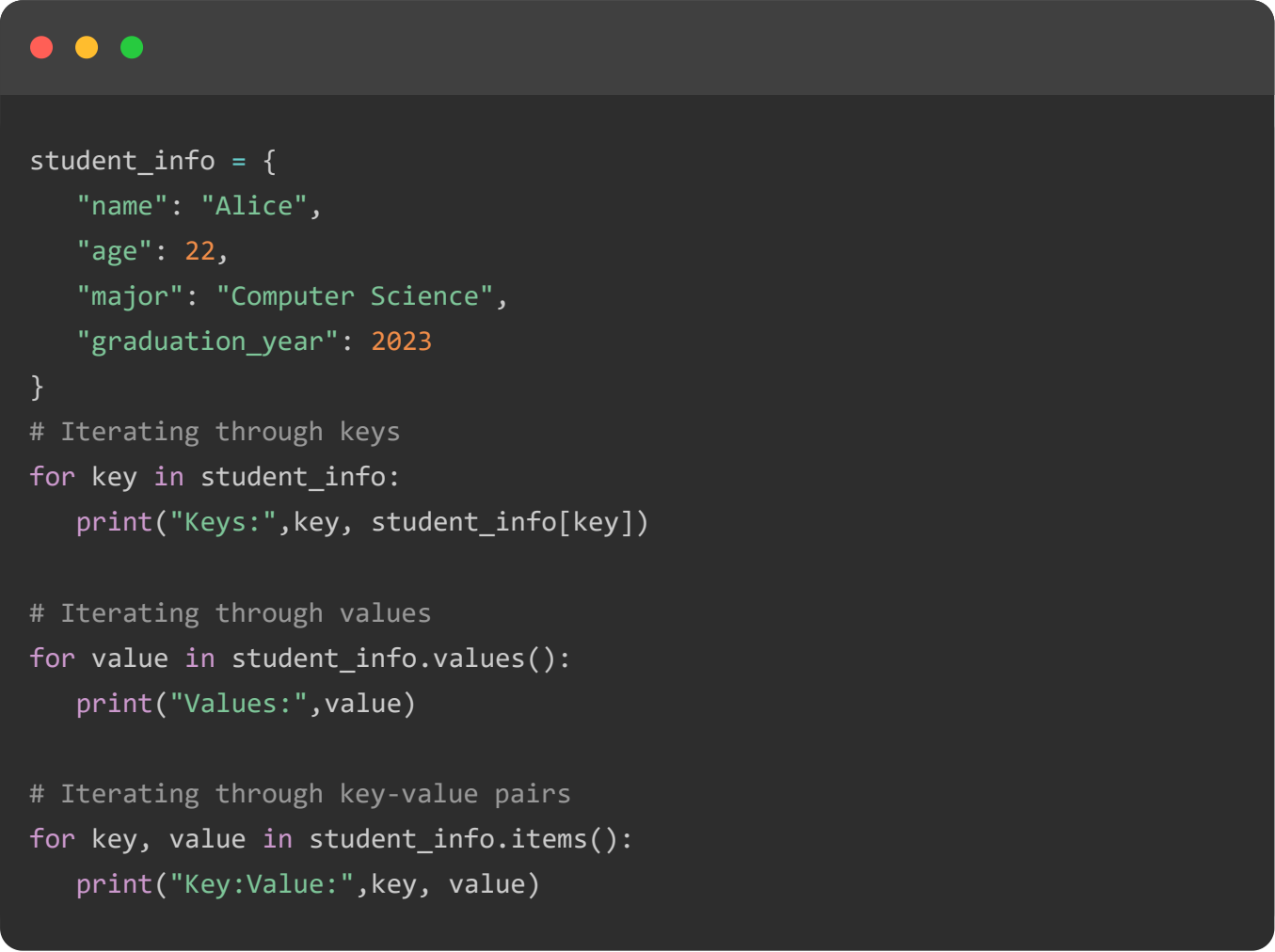
print(student_info)
```

Following is the output of the above code –

```
{'name': 'Alice', 'age': 22}
```

Iterating Through a Dictionary

You can iterate through the keys, values, or key-value pairs in a dictionary using loops –



```
student_info = {
    "name": "Alice",
    "age": 22,
    "major": "Computer Science",
    "graduation_year": 2023
}

# Iterating through keys
for key in student_info:
    print("Keys:",key, student_info[key])

# Iterating through values
for value in student_info.values():
    print("Values:",value)

# Iterating through key-value pairs
for key, value in student_info.items():
    print("Key:Value:",key, value)
```

After executing the above code, we get the following output –

```
Keys: name Alice
Keys: age 22
Keys: major Computer Science
Keys: graduation_year 2023
Values: Alice
```

```

Values: 22
Values: Computer Science
Values: 2023
Key:Value: name Alice
Key:Value: age 22
Key:Value: major Computer Science
Key:Value: graduation_year 2023

```

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys –

- More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example –

```

dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print ("dict['Name']: ", dict['Name'])

```

When the above code is executed, it produces the following result –

```
dict['Name']: Manni
```

- Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example –

```

dict = {['Name']: 'Zara', 'Age': 7}
print ("dict['Name']: ", dict['Name'])

```

When the above code is executed, it produces the following result –

```

Traceback (most recent call last):
  File "test.py", line 3, in <module>

```

```
dict = {'Name': 'Zara', 'Age': 7};
TypeError: unhashable type: 'list'
```

Python Dictionary Operators

In Python, following operators are defined to be used with dictionary operands. In the example, the following dictionary objects are used.

```
d1 = {'a': 2, 'b': 4, 'c': 30}
d2 = {'a1': 20, 'b1': 40, 'c1': 60}
```

Operator	Description	Example
dict[key]	Extract/assign the value mapped with key	print (d1['b']) retrieves 4 d1['b'] = 'Z' assigns new value to key 'b'
dict1 dict2	Union of two dictionary objects, returning new object	d3=d1 d2 ; print (d3) {'a': 2, 'b': 4, 'c': 30, 'a1': 20, 'b1': 40, 'c1': 60}
dict1 =dict2	Augmented dictionary union operator	d1 =d2; print (d1) {'a': 2, 'b': 4, 'c': 30, 'a1': 20, 'b1': 40, 'c1': 60}

Python Dictionary Methods

Python includes following dictionary methods –

Sr.No.	Methods with Description
1	dict.clear() Removes all elements of dictionary dict
2	dict.copy() Returns a shallow copy of dictionary dict
3	dict.fromkeys() Create a new dictionary with keys from seq and values set to value.
4	dict.get(key, default=None) For key key, returns value or default if key not in dictionary
5	dict.has_key(key)

	Returns true if key in dictionary dict, false otherwise
6	dict.items() Returns a list of dict's (key, value) tuple pairs
7	dict.keys() Returns list of dictionary dict's keys
8	dict.setdefault(key, default=None) Similar to get(), but will set dict[key]=default if key is not already in dict
9	dict.update(dict2) Adds dictionary dict2's key-values pairs to dict
10	dict.values() Returns list of dictionary dict's values

Built-in Functions with Dictionaries

Following are the built-in functions we can use with Dictionaries –

Sr.No.	Function with Description
1	cmp(dict1, dict2) Compares elements of both dict.
2	len(dict) Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	str(dict) Produces a printable string representation of a dictionary
4	type(variable) Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

TOP TUTORIALS

Python Tutorial

Java Tutorial

C++ Tutorial

C Programming Tutorial

C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

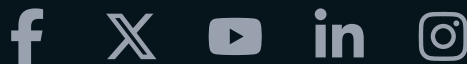
[Online Python Compiler](#)
[Online Go Compiler](#)
[Online C Compiler](#)
[Online C++ Compiler](#)
[Online C# Compiler](#)
[Online PHP Compiler](#)
[Online MATLAB Compiler](#)
[Online Bash Terminal](#)
[Online SQL Compiler](#)
[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Chapters ▾

Categories ≡



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.